



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ N 2

Название: «Разработка усовершенствованного электронного аналога
шифровальной машины Enigma»

Дисциплина: «Защита информации»

Студент ИУ7-72Б
(Группа)

(Подпись, дата)

Ю.А. Попов
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Ю.С. Руденкова
(И. О. Фамилия)

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Анализ предметной области	4
1.1 Принцип работы Enigma	4
1.2 Архивы и потоковая обработка	4
1.3 Требования к контролю целостности	5
2 Проектирование программы	6
2.1 Конфигурация роторной машины	6
2.2 Формат защищенного контейнера	7
2.3 Выборочное шифрование записей	8
3 Реализация интерфейса и криптоанализа	9
3.1 CLI приложения	9
3.2 Атака по известному открытому тексту	9
4 Тестирование	11
4.1 Методика тестирования	11
4.2 Результаты	11
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	14

ВВЕДЕНИЕ

Роторная машина Enigma представляет интерес как исторический пример поточного шифра: результат преобразования зависит от текущего состояния роторов, которое изменяется после каждого символа. Историческая конструкция использовала алфавит из 26 букв и не предназначалась для обработки двоичных файлов. В настоящей работе механизм обобщен на алфавит из 256 значений байта и дополнен средствами, необходимыми для работы с архивными данными.

Цель работы — разработать усовершенствованный электронный аналог шифровальной машины Enigma с потоковым шифрованием архивов, проверкой целостности и демонстрацией известного криптоанализа.

Для достижения цели необходимо решить следующие задачи:

- 1) реализовать конфигурируемую машину с коммутационной панелью, выбором трех роторов из четырех и выбором отражателя;
- 2) реализовать потоковое шифрование и расшифрование ZIP и TAR архивов без загрузки архива целиком в память;
- 3) добавить аутентификацию шифротекста и защиту файла назначения от неверного ключа;
- 4) обеспечить преобразование отдельных файлов внутри архива без распаковки архива в каталог;
- 5) создать CLI с режимами шифрования и расшифрования;
- 6) реализовать базовую атаку по известному открытому фрагменту архива.

1 Анализ предметной области

1.1 Принцип работы Enigma

В классической машине сигнал проходит через коммутационную панель и роторы справа налево, отражается и возвращается через те же роторы в обратном направлении. Отражатель попарно соединяет контакты, поэтому путь сигнала симметричен. При одинаковом начальном состоянии одна и та же операция выполняет как шифрование, так и расшифрование [1].

Перед обработкой очередного символа правый ротор делает шаг. При достижении точки переключения он сдвигает средний ротор; средний ротор в своей точке переключения сдвигает также левый. Это двойной шаг, характерный для Enigma [2].

Историческая Enigma не является криптографически стойкой системой. В частности, известный открытый текст и повторяемые служебные фрагменты сообщений позволяют уменьшить пространство поиска настроек. Поэтому в работе машина рассматривается как учебная модель, а HMAC добавлен для контроля целостности, а не как утверждение о современной криптографической стойкости.

1.2 Архивы и потоковая обработка

ZIP хранит каталог записей и может сжимать каждую запись отдельно; TAR последовательно хранит заголовки и содержимое файлов. Библиотеки стандартной библиотеки Go предоставляют потоковые интерфейсы для работы с обоими форматами [3] [4]. Полная загрузка архива в память для больших файлов недопустима. Поэтому чтение, преобразование и запись выполняются блоками по 64 KiB.

Для шифрования всего архива введен контейнер ENIGAR1. Он сохраняет исходный архив как непрерывный поток шифротекста, благодаря чему его внутренняя структура, включая каталоги и метаданные, остается без изменений после расшифрования. Выборочное преобразование записей требует пересборки ZIP или TAR, но содержимое каждой записи при этом также копируется потоково, без извлечения во временный каталог.

1.3 Требования к контролю целостности

Потоковое шифрование не должно публиковать расшифрованный архив до проверки его подлинности. Иначе неверная начальная установка роторов создаст поврежденный ZIP или TAR с правдоподобными байтами. Для этой цели используется код аутентификации HMAC SHA 256 из стандартной библиотеки Go [5]. Сравнение тегов производится в постоянном времени. Выход записывается во временный файл в каталоге назначения и только после успешной проверки атомарно переименовывается.

2 Проектирование программы

2.1 Конфигурация роторной машины

Машина использует четыре активных 256 контактных ротора. Проводка каждого ротора является перестановкой значений от 0 до 255; для быстрого обратного прохода хранится также обратная перестановка. В конфигурации всегда присутствуют роторы I, II, III, IV; пользователь задает их порядок установки. Доступны два отражателя A и B.

Коммутационная панель задается набором непересекающихся пар, например 65:66, 0x01:0xff. Неуказанные значения отображаются сами в себя. Пара не может содержать одинаковые байты и не может пересекаться с другой парой. Основные объекты представлены в Таблица 2.1.

Таблица 2.1 — Основные структуры реализации

Объект	Поле	Назначение
Ротор	wiring, reverse	Прямая и обратная перестановки 256 байтов.
Ротор	notch, position	Точка переключения и текущее смещение.
Конфигурация	Rotors	Четыре идентификатора роторов в порядке установки.
Конфигурация	Reflector, Positions	Отражатель и начальные позиции.
Машина	plugboard	Попарное отображение коммутационной панели.
Машина	key	Хеш канонической конфигурации для HMAC.

Обозначим преобразование панели через P , прямой проход роторов через R_r, R_m, R_l , а отражатель через U . Преобразование i го байта имеет вид

$$C_i = P \left(R_r^{-1} \left(R_m^{-1} \left(R_l^{-1} \left(R_f^{-1} \left(U \left(R \left(f \left(R \left(l \left(R_m \left(R_r \left(P(M_i) \right) \right) \right) \right) \right) \right) \right) \right) \right) \right) \right) \right) \right) \quad (2.1)$$

Все отображения обратимы, а P и U являются собственными обратными отображениями. Следовательно, новая машина с теми же настройками преобразует C_i обратно в M_i .

Конфигурация может быть передана флагами или JSON файлом. Пример файла приведен в Листинг 2.1.

```
1 {  
2   "rotors": ["IV", "II", "I", "III"],  
3   "reflector": "B",  
4   "positions": "7,12,34,56",  
5   "plugboard": ["65:66", "0x01:0xff"]  
6 }
```

Листинг 2.1 — Пример конфигурационного файла машины

2.2 Формат защищенного контейнера

Контейнер состоит из фиксированного заголовка размером 36 байтов, шифротекста и 32 байтового тега. Заголовок содержит сигнатуру ENIGAR1, версию, тип исходного архива, размер исходного потока и первые 16 байтов отпечатка конфигурации. Отпечаток позволяет отказаться от расшифрования с явно неподходящей конфигурацией до создания результата.

Тег вычисляется по заголовку H и шифротексту C :

$$T = \text{HMAC-SHA-256}(K_a, H \parallel C), \quad (2.2)$$

где K_a — ключ, полученный хешированием канонического представления настроек машины; $\parallel$ — конкатенация. При расшифровании в HMAC сначала поступают исходные байты шифротекста, затем они преобразуются и записываются во временный файл. Это сохраняет потоковый характер обработки.

Схема полного архивного режима показана в Листинг 2.2.

```
1 ZIP/TAR -> read 64 KiB -> Enigma -> write ciphertext -> HMAC -> ENIGAR1  
2 ENIGAR1 -> verify fingerprint -> read 64 KiB -> Enigma -> temporary file  
3                                     -> verify HMAC -> rename
```

Листинг 2.2 — Поточковая обработка контейнера ENIGAR1

2.3 Выборочное шифрование записей

Команда `entries` получает перечень имен файлов. Для ZIP каталог читается из исходного файла, затем каждая запись открывается, при необходимости преобразуется блоками и записывается в новый ZIP. Для TAR записи читаются последовательно и сразу передаются в новый TAR. Каталоги, пустые файлы и невыбранные записи копируются без преобразования.

В результирующий архив добавляется служебная запись `.enigma-auth.json`. Она содержит перечень преобразованных файлов и HMAC логического содержимого архива. Перед выборочной расшифровкой программа делает отдельный потоковый проход для проверки манифеста. Лишь затем создается архив результата. Таким образом, неверная настройка не повреждает метаданные архива или файл назначения.

3 Реализация интерфейса и криптоанализа

3.1 CLI приложения

Для построения CLI использована библиотека Cobra [6]. Поддерживаются явные команды `encrypt` и `decrypt`, а также короткие режимы `-e` и `-d`. Основные параметры приведены в Таблица 3.1.

Таблица 3.1 — Интерфейс командной строки

Параметр или команда	Назначение
<code>-e, encrypt</code>	Шифрование всего входного ZIP или TAR в контейнер ENIGAR1.
<code>-d, decrypt</code>	Проверка HMAC и расшифрование контейнера.
<code>entries --files</code>	Преобразование только перечисленных записей внутри архива.
<code>--rotors, --reflector</code>	Выбор порядка роторов и отражателя.
<code>--positions, --plugboard</code>	Начальные позиции и пары коммутационной панели.
<code>--config</code>	Путь к JSON файлу конфигурации.
<code>attack</code>	Перебор одной неизвестной пары по известному открытому фрагменту.

Пример шифрования архива приведен в Листинг 3.1.

```
1 go run ./cmd/enigma -e -i backup.zip -o backup.egar \  
2 --rotors IV,II,I,III --reflector B --positions 7,12,34,56 \  
3 --plugboard 65:66,0x01:0xff
```

Листинг 3.1 — Пример запуска шифрования

3.2 Атака по известному открытому тексту

Для демонстрации уязвимости реализована ограниченная атака на одну неизвестную пару коммутационной панели. Предполагается, что атакующему известны порядок роторов, отражатель и начальные позиции, но неизвестна одна пара $a : b$. Также известен фрагмент открытого архива M и соответствующий фрагмент шифротекста C . В качестве такого фрагмента может использоваться заголовок ZIP, начинающийся с PK, либо фрагмент `ustar` в TAR.

Алгоритм перебирает все $C(256, 2) = 32640$ неупорядоченных пар байтов. Для каждой пары создается машина с этой единственной парой, роторы продви-

гаются на смещение известного фрагмента и выполняется преобразование M . Пара остается кандидатом, если полученный результат совпадает с C . Чем длиннее известный фрагмент, тем меньше кандидатов остается. Это не полный взлом Enigma, но практическая иллюстрация частичного восстановления настройки панели.

В проект включен воспроизводимый сценарий `scripts/demo_attack.sh`. Он создает временный ZIP, шифрует его с секретной парой 65:66, передает атаке первые 128 байтов известного исходного ZIP и проверяет, что 65:66 находится среди найденных кандидатов. Фрагмент запуска приведен в Листинг 3.2.

```
1 $ ./scripts/demo_attack.sh
2 Найдена искомая пара: 65:66 (кандидатов после фильтрации: 1).
3 Успех: известный фрагмент сузил поиск и восстановил часть настройки панели.
```

Листинг 3.2 — Результат демонстрации атаки известного текста

4 Тестирование

4.1 Методика тестирования

Автоматические тесты написаны для пакетов `internal/enigma` и `internal/archive`. Набор проверок охватывает обратимость роторного преобразования, ошибочные позиции, полный контейнерный режим и выборочную обработку ZIP и TAR. Сценарии приведены в Таблица 4.1.

Таблица 4.1 — Набор тестовых сценариев

N	Проверяемое условие	Ожидаемый результат
1	Повторное преобразование 512 байтов с одинаковой конфигурацией.	Исходный массив восстановлен побайтно.
2	Позиция ротора 256.	Возвращается ошибка диапазона.
3	ZIP в контейнере ENIGAR1.	Расшифрованный архив совпадает с исходным побайтно.
4	Расшифрование с другой позицией ротора.	Возвращается ошибка отпечатка или HMAC; прежний файл назначения сохранен.
5	Выборочная запись ZIP, включая пустую запись.	Выбранный файл восстановлен; пустой файл и структура сохранены.
6	Выборочная запись TAR.	Содержимое выбранного файла восстановлено потоково.
7	Сценарий <code>demo_attack.sh</code> .	Фактическая пара 65:66 присутствует в множестве кандидатов.

Проверка выполняется командами из корня каталога `lab_02`:

```
1 go test ./...
2 go vet ./...
3 ./scripts/demo_attack.sh
```

Листинг 4.1 — Команды проверки программы

4.2 Результаты

Модульные тесты успешно подтверждают обратимость машины для бинарных данных и корректность обработки граничного положения 255. Интеграционные тесты подтверждают, что ZIP и TAR обрабатываются без полного чтения в память, а пустые записи не нарушают последовательность обработки.

Проверка неверной конфигурации завершается до публикации расшифрованного архива. Это демонстрирует выполнение требования защиты метаданных: файл назначения сохраняет прежнее содержимое, поскольку временный файл удаляется. Скрипт атаки завершился успешно; известный открытый фрагмент позволил включить секретную пару в существенно меньшее, чем полное, пространство настроек панели.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы разработан усовершенствованный электронный аналог Enigma для алфавита из 256 байтов. Реализованы четыре активных ротора, настраиваемый порядок их установки, два отражателя, коммутационная панель и загрузка параметров из JSON.

Для ZIP и TAR реализованы два потоковых режима: защищенный контейнер для всего архива и выборочное преобразование записей внутри архива. Обработка выполняется блоками по 64 KiB. Контейнер использует HMAC SHA 256, а расшифрованный результат публикуется только после проверки тега, что предотвращает повреждение файлов при неверных настройках.

Создан CLI с флагами -e и -d, конфигурационными параметрами и отдельной командой атаки. Сценарий известного открытого текста демонстрирует частичное восстановление настройки коммутационной панели, подтверждая учебную криптографическую уязвимость модели Enigma.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. National Security Agency. Outline of Enigma Machines [Электронный ресурс]. — 2026. — Режим доступа: https://www.cryptomuseum.com/crypto/enigma/files/enigma_wiring_catalog.pdf.
2. Hamer D. H. Enigma: Actions Involved In Double Stepping [Электронный ресурс]. — 2026. — Режим доступа: https://www.cryptomuseum.com/people/hamer/files/double_stepping.pdf.
3. The Go Authors. archive/zip package [Электронный ресурс]. — 2026. — Режим доступа: <https://pkg.go.dev/archive/zip>.
4. The Go Authors. archive/tar package [Электронный ресурс]. — 2026. — Режим доступа: <https://pkg.go.dev/archive/tar>.
5. The Go Authors. crypto/hmac package [Электронный ресурс]. — 2026. — Режим доступа: <https://pkg.go.dev/crypto/hmac>.
6. The Cobra Authors. cobra package [Электронный ресурс]. — 2026. — Режим доступа: <https://pkg.go.dev/github.com/spf13/cobra>.